

Architecting reproducible science: Best Scientific Computing Practices

Software Engineering principles to improve your HPC

Fernando Garzon, August 2026

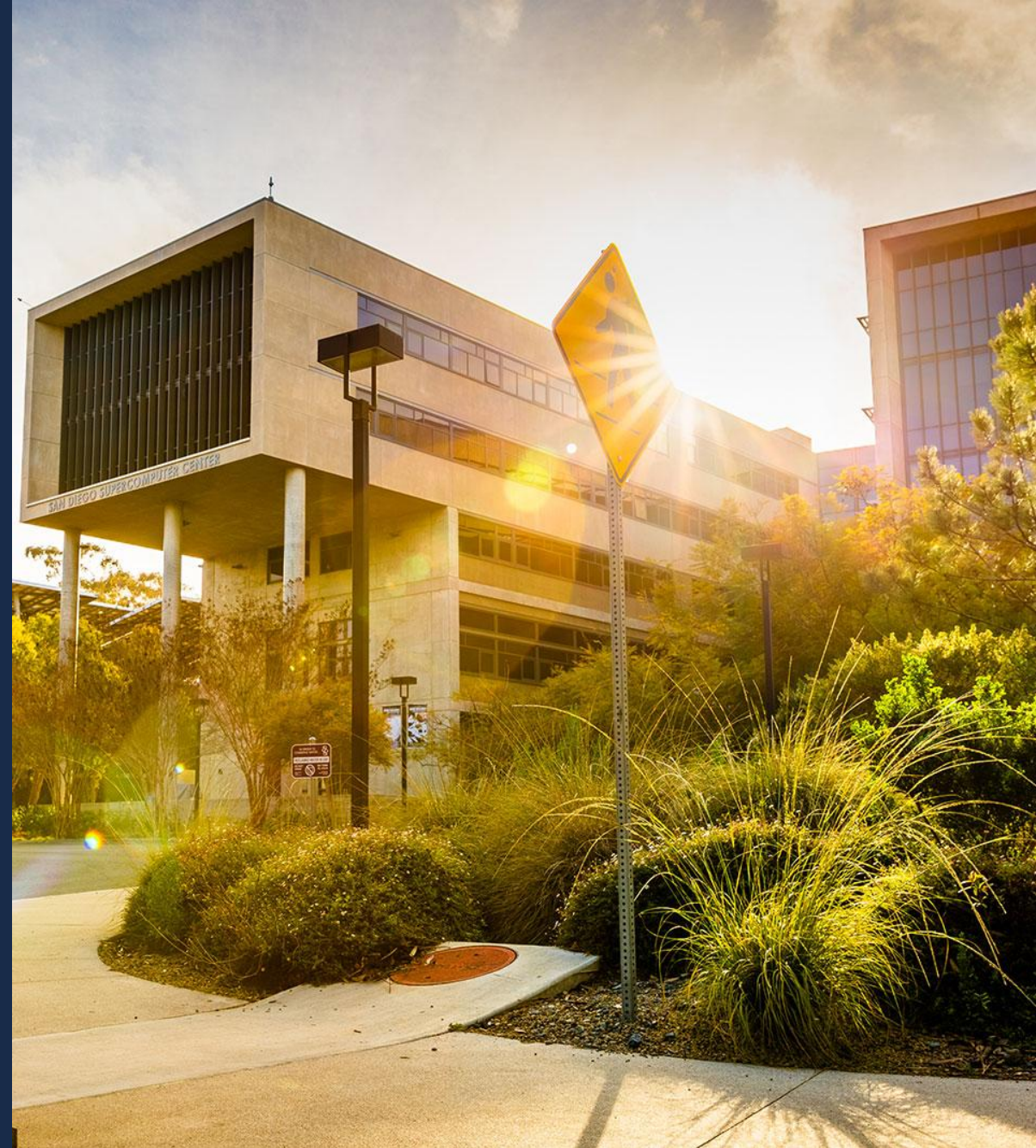
[linkedin.com/in/fgarzonm](https://www.linkedin.com/in/fgarzonm) - github.com/ofgarzon2662

SDSC
SAN DIEGO SUPERCOMPUTER CENTER

UC San Diego

Who's Fernando Garzon

- Software Engineer
- Computational & Data Scientist Research Specialist at SDSC
- Work on User Support for TSCC.
- Also work in Software Dev (SWD) projects within SDSC.
- Passionate about SWD, DevOps, Automation, Cloud Architecture, IaC and more.
- Automation Enthusiast.

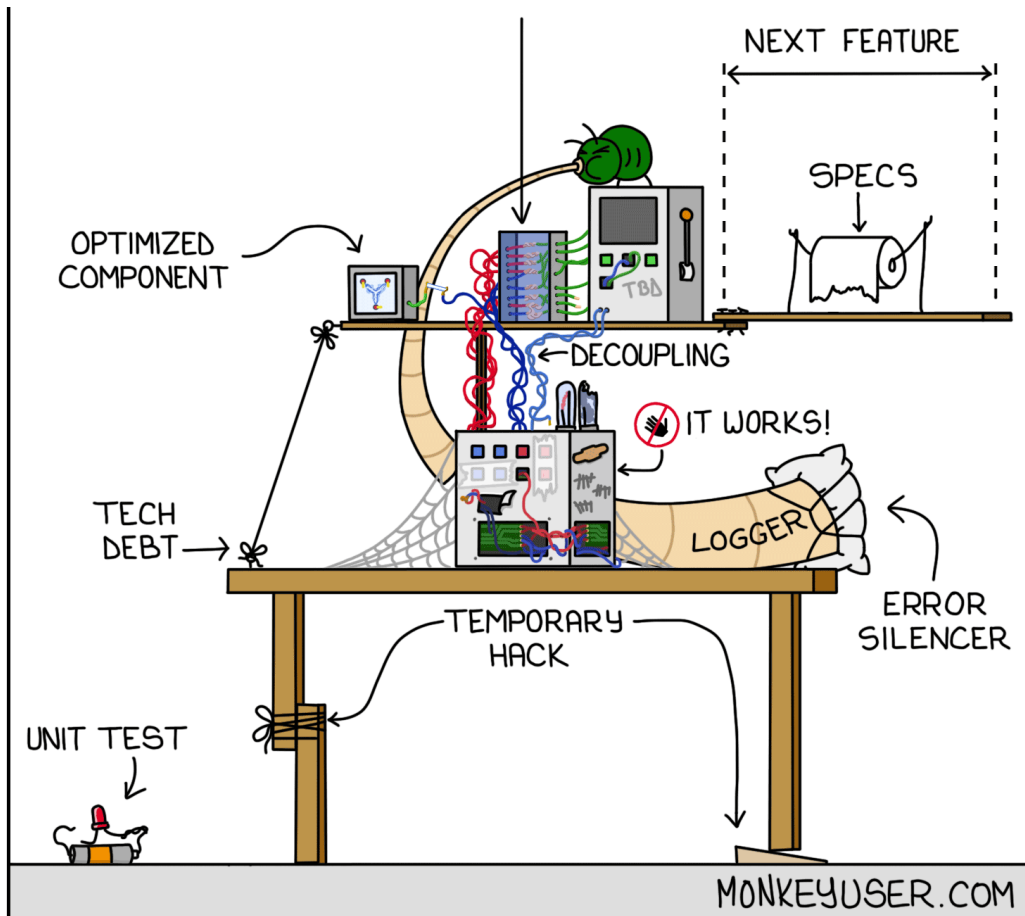




Outline

1. Intro: The un-reproducibility horror
2. Notebooks Don't Scale. Packages Do.
3. Let's Prod this.
4. Lights, Camera, GitHub Actions!
5. MLOps – Turning your Notebook into a Money-Printing Machine.
6. Final remarks

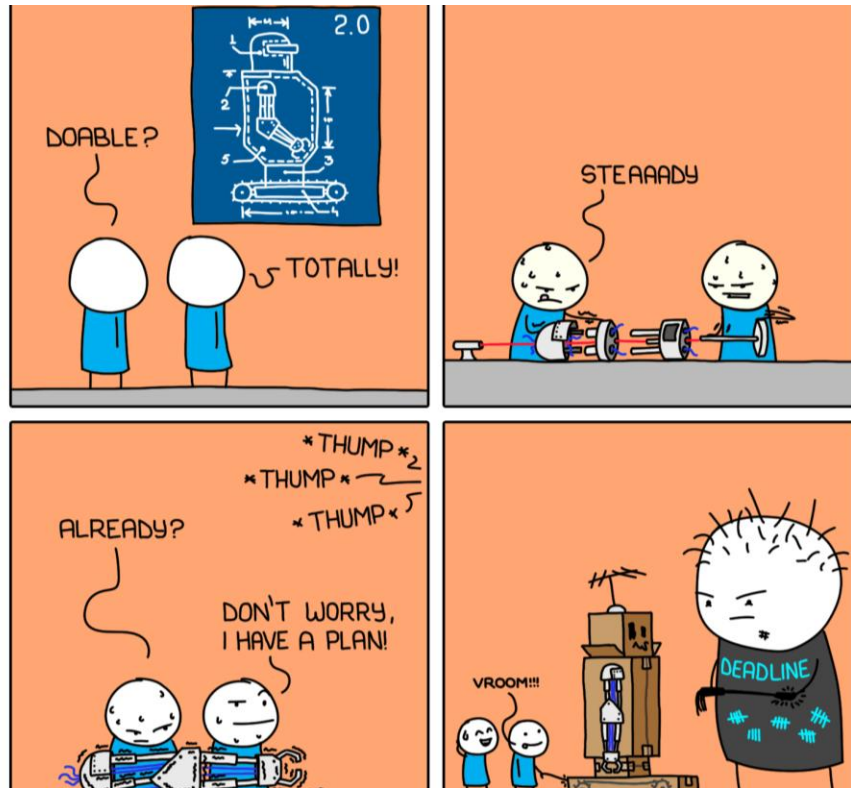
Intro: The un-reproducibility horror



- Milena's model in the Jpy NB worked perfectly. It ingested and analyzed the data, generated the proper outputs, and most importantly, it promised a big balance in her and the stakeholders' bank accounts.
- The mess up: She forgot to run an apparently meaningless cell, one that optimized the routine to prevent memory overload. Fans roared and laptop froze.
- Results: Terrible. Stakeholders who know nothing about code and care less about it decided this isn't worth the headache.
- Somehow, when it mattered most—in front of the stakeholders—Milena was unable to **reproduce** the strong results her model had previously achieved.
- The lesson: There's just gotta be a better way...and there is.

Intro video: <https://www.youtube.com/watch?v=5AK7SKgkjms>

Notebooks Don't Scale. Packages Do*

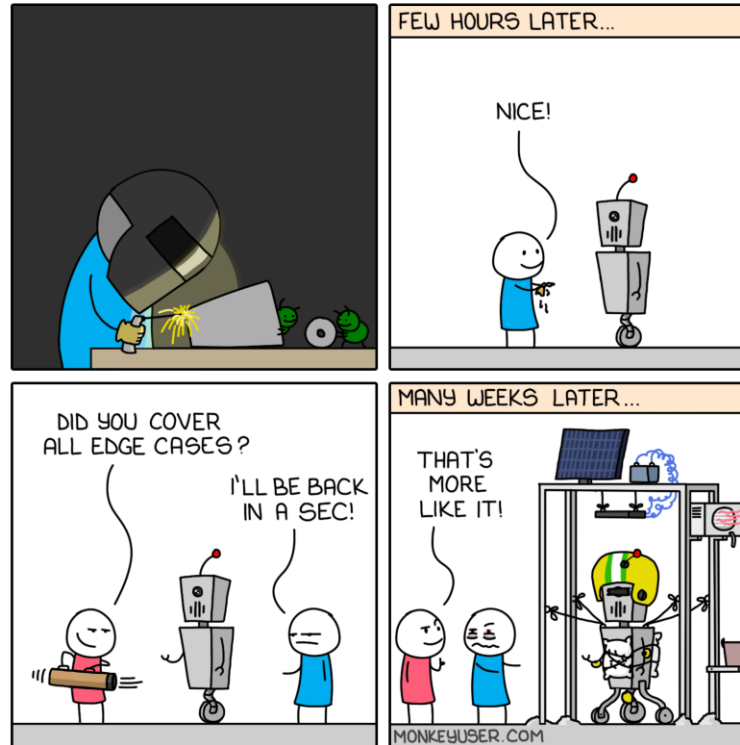


* not exactly, NBs can scale, just not as good as Modular Packages do.

- What if you could deploy your code on any laptop/server/system/cluster/container/cloud provider and still get the same predictable results? Is the code **Portable**?
- What if anyone could rerun your code on a clean environment and obtain the same results. Are your code and results **Repeatable**, is the workflow producing consistent and deterministic results, without the worry that it will 'skip a cell' in the process?
- What if you could run simple, fast, cheap tests to verify your code does what's supposed to, without having to 'run all the cells' or the full implementation? Is your code **Testable**?
- What if your code could run across hundreds of servers, support hundreds of concurrent users, and handle larger datasets or more compute resources while still meeting requirements for performance, latency, memory, cost, reliability, and security? Is it **Scalable**?

Notebooks Don't Scale. Packages Do*

EDGE CASES



* not exactly, NBs can scale, just not as good as Modular Packages do.

- NBs can do all that and devs work on that every day: NBs can be **Scalable**.
- NBs are also amazing for exploratory work: You got an idea, you test it on the fly, you see instant results/plots/numbers just one 'shift' + 'enter' hit away.
- However, and for the *same* reason, if good coding practices aren't kept (and *most* people in academia don't), NBs can grow out of control quickly with messed up code all over the cells.
- And AI just facilitates the creation of big messed up code bases faster.
- So yeah, NBs are great, but hard to maintain and **Scale**.
- So, what's the other way?

Modular Python Packages

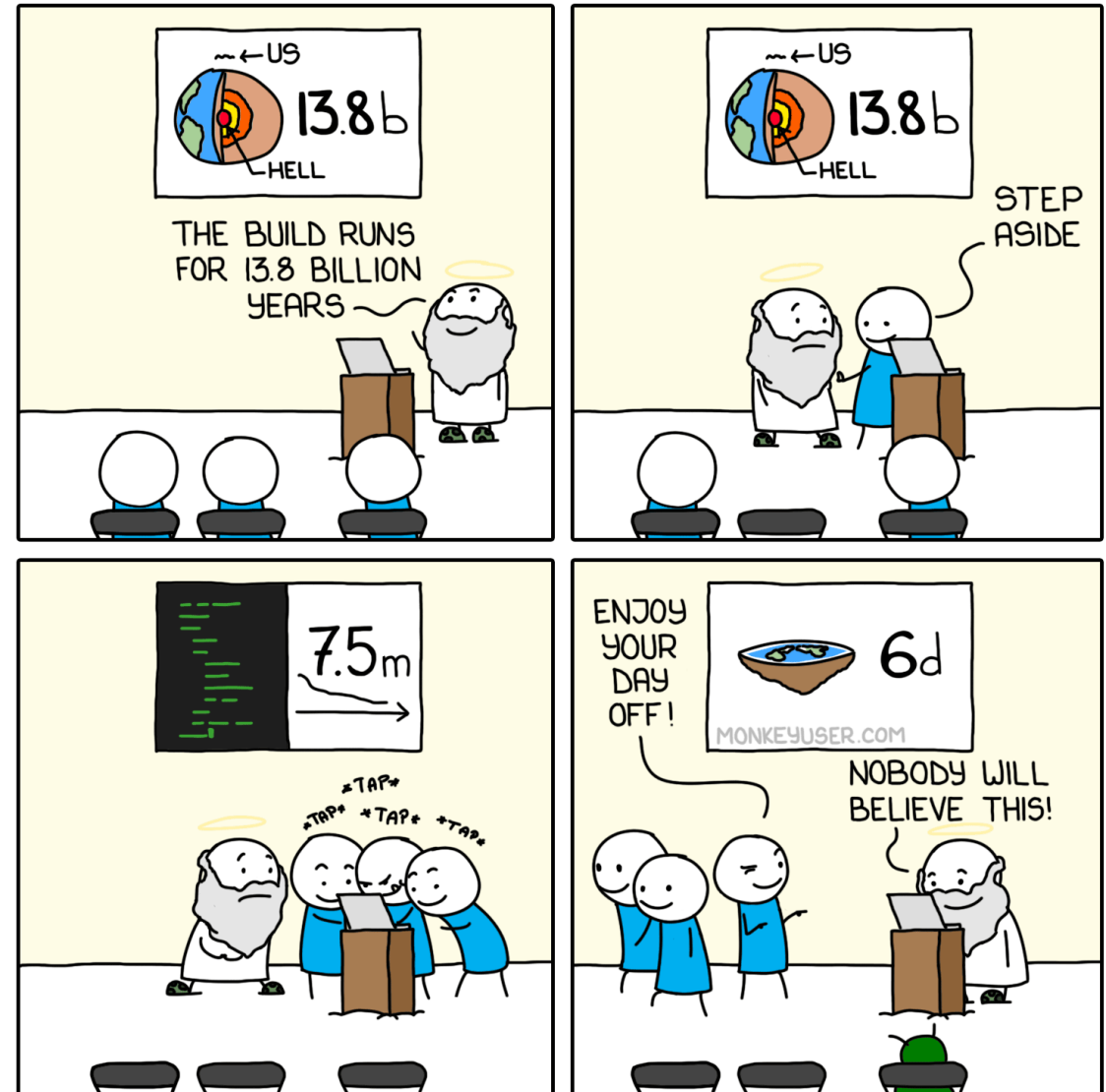
Let's Prod this

- *Modular Python Packages (MPyP)*: They Scale real nice.
- **IF** coded properly, they're small, manageable, Testable, easy to deploy, can run anywhere, and best of all, they're **Scalable**.
- They are like little bricks that you can handle easily with your own hands, instead of trying to carry the whole building around wherever you go.
- Therefore, the strategy is clear: We need to learn to migrate our Jupyter Notebooks to MPyPs

Follow the instructions in here:

https://github.com/ofgarzon2662/mnist_ae/blob/master/README.md

NEW WORLD



Let's Prod this

Prompt Engineering and AI assisted development. Use SOLID principles:

You are a senior Python engineer. Rewrite the file ``mnist_ae/mnist_training.py`` so that:

- *Each class/function has one clear responsibility (Single-Responsibility Principle).*
- *Related functionality is grouped into modules (e.g. `data`, `model`, `training`, `cli`).*
- *Internal helpers are made private (`_`prefix).*
- *No global execution at import-time; provide a ``main()`` entry point.*
- *Add type hints and docstrings.*

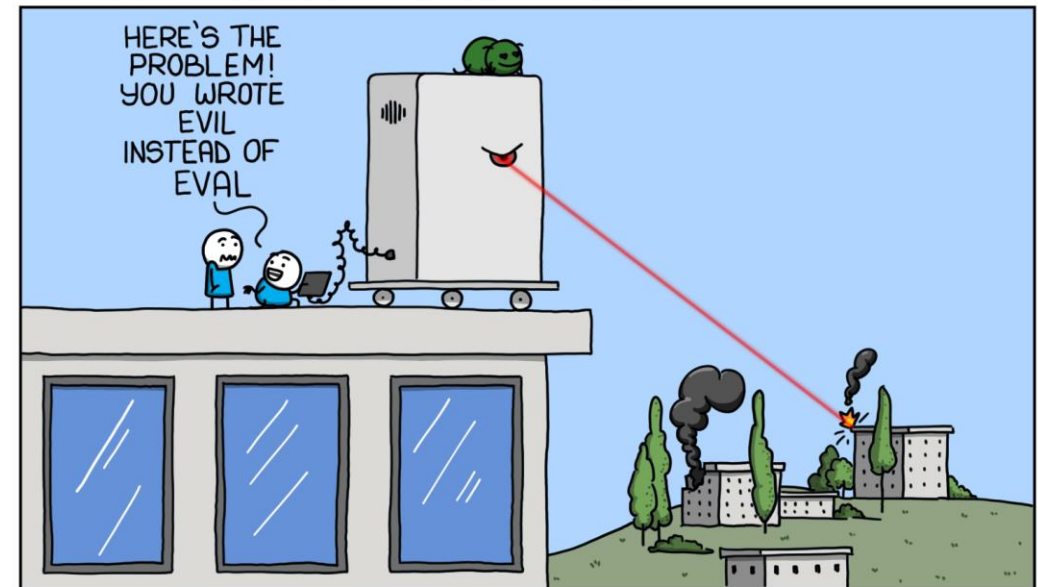
Return the full, refactored code as a valid Python package structure.

Follow the instructions in here:

https://github.com/ofgarzon2662/mnist_ae/blob/master/README.md

NATURAL LANGUAGE INSTRUCTIONS

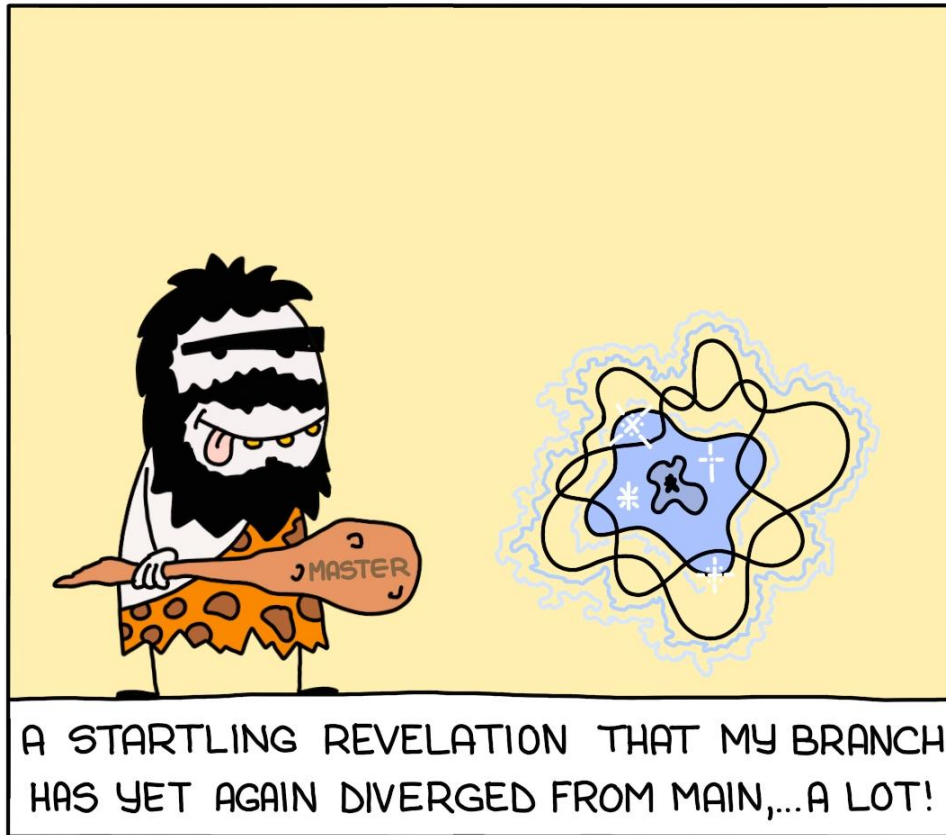
MONKEYUSER.COM



Lights, Camera, GitHub Actions!

RAMIFICATIONS

MONKEYUSER.COM



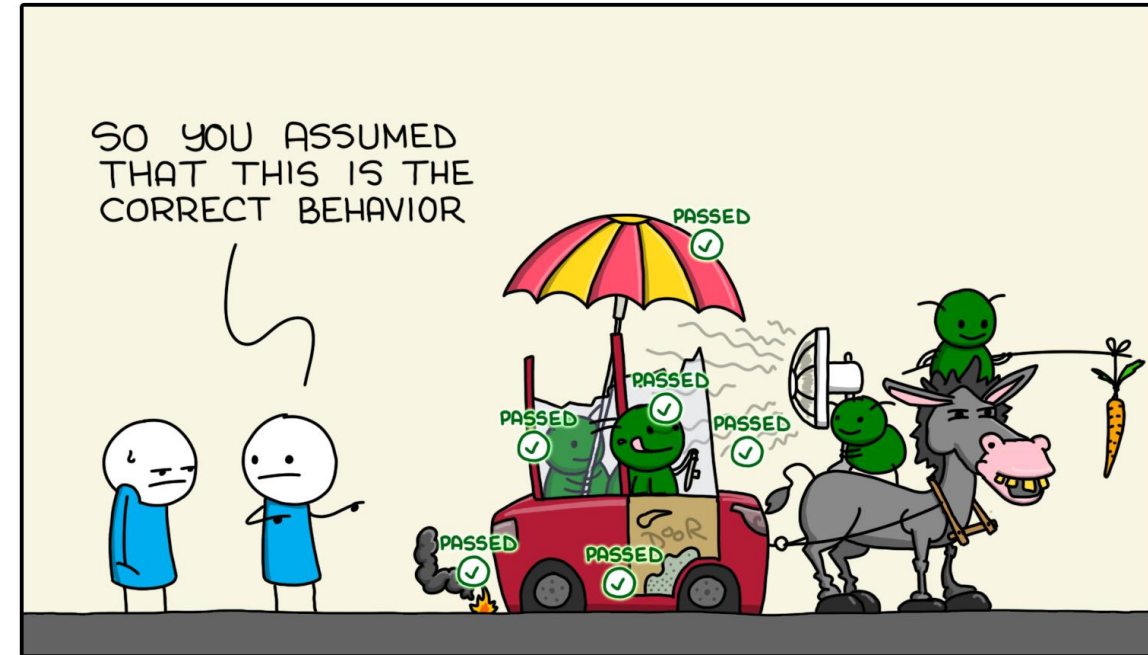
- DevOp\$ is a seriously promising profitable field of work. And it's been for the last 15 years.
- Combining DevOps with AI tools can make you super attractive to potential employers in high tech industries and HPC.
- DevOps is the art of automating deployments in a fast, controlled, and a reproducible manner.
- And, GitHub Actions are the 1st step in the DevOps stairs.
- Easy to implement, free or super cheap at worst, and they should be mandatory on every project you work on starting today.

Lights, Camera, GitHub Actions!

- The most basic automation one should always do is Unit Testing and code coverage.
- But you can extend your Github actions to cover Security, Linting, basic integration tests, basic deployment tests in dedicated VMs or envs, basic smoke tests.
- And, you can also implement Github actions to automate deployments of Python Packages.

UNIT TESTS

MONKEYUSER.COM

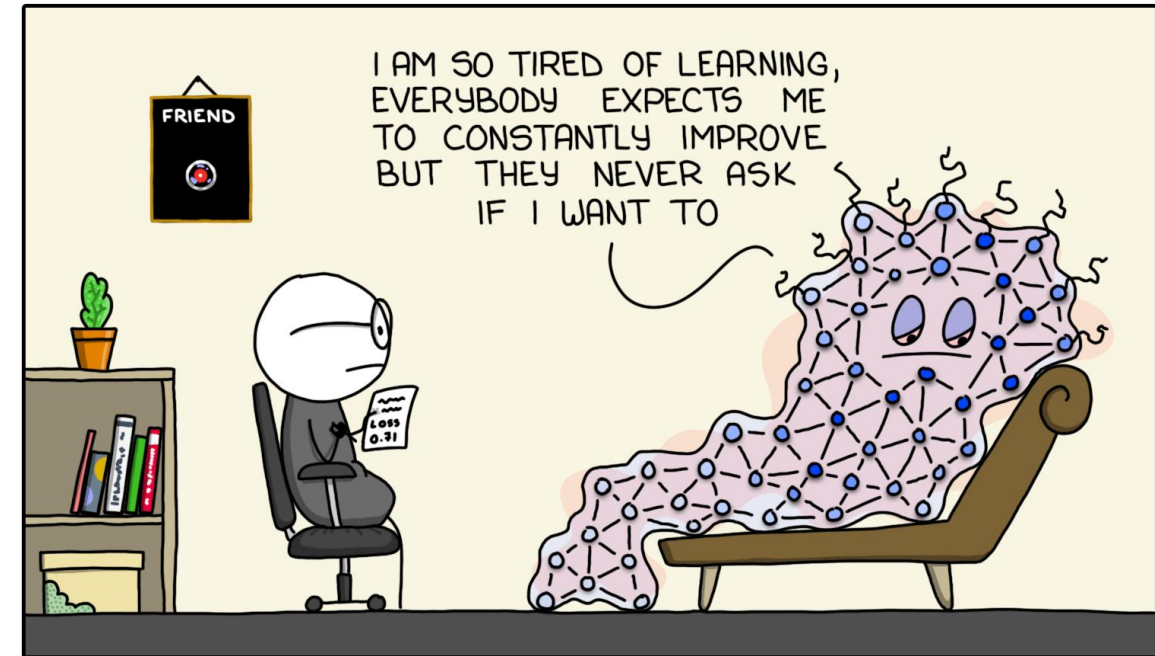


MLOps: Turning your Notebook into a Money-Printing Machine.

- MLOps applies DevOps practices to machine-learning systems.
- It makes code, data, models, tests, releases, and monitoring more repeatable.
- The goal is reliable model behavior from experiment through deployment.
- Your ML expertise becomes much more useful when you can test, version, deploy, and operate the system around the model.
- Those are concrete, portfolio-ready skills that employers recognize.

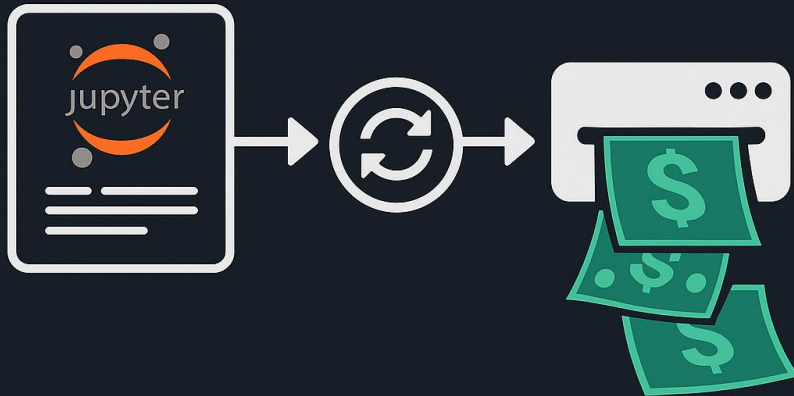
AI CARE

MONKEYUSER.COM



MLOps: Turning your Notebook into a Money-Printing Machine.

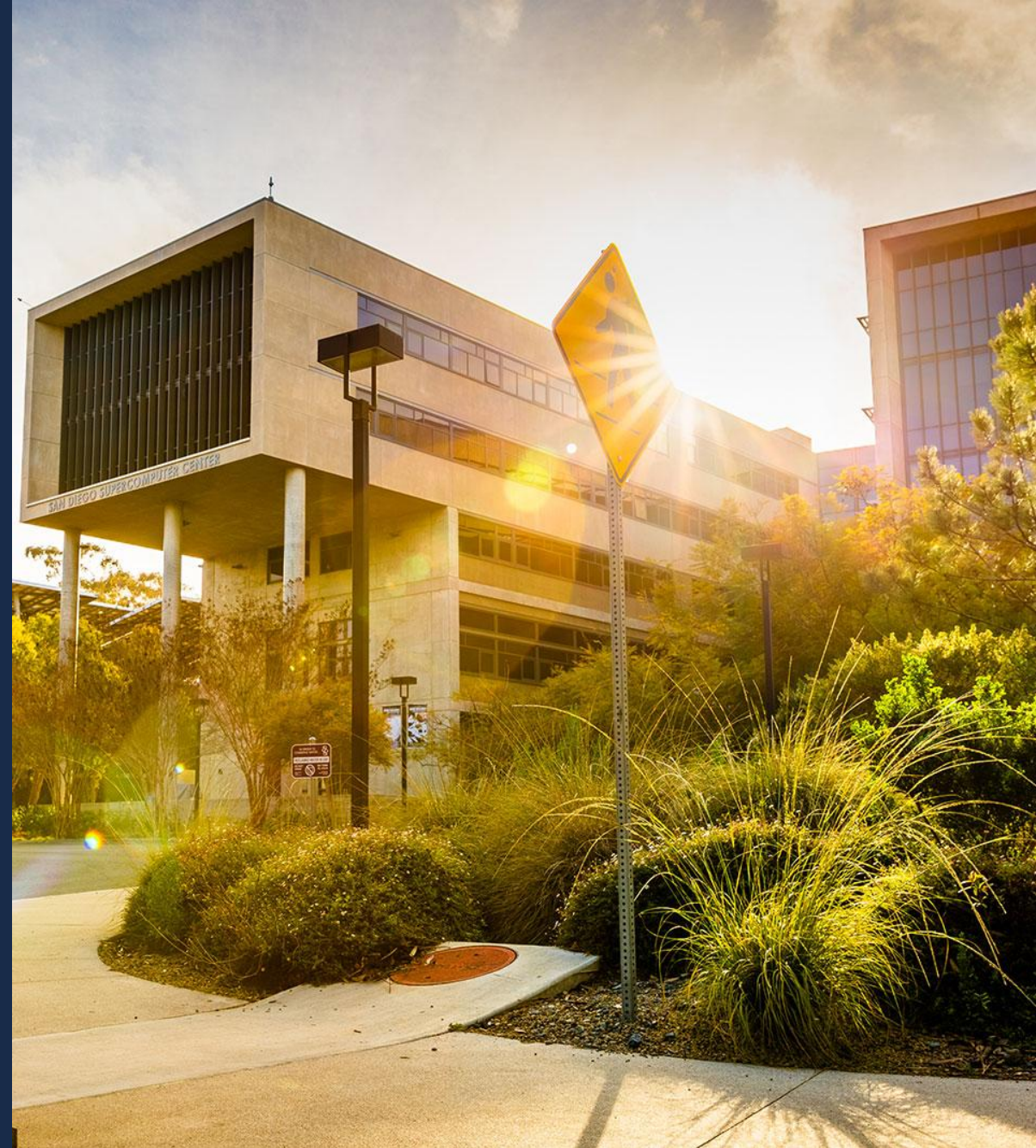
MLOps: Turning your Notebook into a Money-Printing Machine



1. Amazon: 136k Deps. a day.
2. Google: $O(10k)$ Deps a day.
3. Netflix: 500 Deps. a day.
4. Etsy: 50 Deps. a day.
5. Meta: 1 Dep. a day.

Final Remarks

1. There are bad ML/AI Engineers, and then there are ML/AI Engineers who can deploy their code to prod.
2. Don't commit big resources and time into testing ML/AI models. Use automated Unit Test first.
3. Github Actions are the 1st step in the automation process but there is plenty more you can learn to leverage your computing practices: (CI/CD, DevOps, MLOps).
4. AI won't take your AI Engineering job. Someone who knows how to use DevOps pipelines to streamline the deployment of AI/ML models into production and generates absurd amounts of revenue, that person will.





Questions



[linkedin.com/in/fgarzonm](https://www.linkedin.com/in/fgarzonm)



github.com/ofgarzon2662